

ГУАП

КАФЕДРА № 42

ОТЧЕТ
ЗАЩИЩЕН С ОЦЕНКОЙ
ПРЕПОДАВАТЕЛЬ

ассистент

должность, уч. степень, звание

подпись, дата

И. А. Юдин

инициалы, фамилия

ОТЧЕТ О ЛАБОРАТОРНОЙ РАБОТЕ № 1

МОДЕЛИРОВАНИЕ БАЗОВОЙ СЛУЧАЙНОЙ ВЕЛИЧИНЫ

по курсу:

МОДЕЛИРОВАНИЕ СИСТЕМ

РАБОТУ ВЫПОЛНИЛ

СТУДЕНТ гр. № 4326

подпись, дата

Г. С. Томчук

инициалы, фамилия

Санкт-Петербург 2026

1 Цель работы

Цель работы: построить датчик базовой случайной величины по заданному алгоритму и выполнить тестирование датчика на соответствие основным свойствам базовой случайной величины.

2 Задание

1. Построить датчик БСВ с периодом $T > 500$.
2. Оценить математическое ожидание и дисперсию псевдослучайных значений z_i и сравнить их с теоретическими значениями M и D .
3. Проверить датчик БСВ на равномерность и построить гистограмму распределения относительных частот $p_1, \dots, p_k$ на K отрезках интервала $[0,1]$.
4. Проверить датчик БСВ на независимость, определяя коэффициент корреляции для разных значений s и T . Построить в одном графическом окне графики зависимости $\hat{R} = f(T)$ для $s = 2, s = 5, s = 10$.

Вариант: № 2 «Генератор “little-frog”». На рисунке 1 изображено задание по варианту.

2 Генератор «little-frog»:

$$A_0 = 1, A_i = A_{i-1}M \bmod (2^r);$$

$$z_i = A_i 2^{-r},$$

где r – число двоичных разрядов, используемых для представления числа в компьютере;

M – множитель, достаточно большое число, взаимно-простое с 2^r ;

z_i – БСВ.

Длина периода T последовательности $\{z_i\}$ равна 2^{r-2} .

Последовательность A_i предварительно разбивается на подпоследовательности длины m , начинающиеся с чисел A_{km} , $k = 0, 1, 2, \dots$ (то есть с начальных значений последовательностей) и каждая подпоследовательность используется для построения соответствующих случайных чисел $A_{(k+1) \cdot m} = A_{km} \cdot M^m \bmod (2^r)$.

Для моделирования k -й подпоследовательности $z_{km} = A_{km} \cdot 2^{-r}$

Множитель M для генератора, обеспечивающего "прыжки" с шагом длиной m , вычисляется на основе равенства $M \equiv M^m \bmod (2^r)$.

Рисунок 1 — Вариант задания

3 Ход выполнения

В ходе выполнения лабораторной работы был реализован датчик базовой случайной величины на языке Python с использованием генератора

«little-frog». В генераторе последовательность разбивается на подпоследовательности длины m . Для перехода между начальными элементами подпоследовательностей используется механизм «прыжков». Прыжковый множитель вычислялся по формуле:

$$M_{jump} = M^m \pmod{2^r},$$

где M — множитель генератора, m — длина подпоследовательностей, r — количество двоичных разрядов представления чисел.

В работе выбраны значения $M = 65539, m = 3, r = 16$. Тогда:

$$M_{jump} = 65539^3 \pmod{65536} = 27.$$

Полученный множитель является нечетным и взаимно простым с 2^r , что обеспечивает корректную работу генератора. После этого генерация выполнялась по рекуррентному соотношению:

$$A_0 = 1, A_i = A_{i-1} \cdot M_{jump} \pmod{2^r}$$

где $A_0 = 1$ — начальное значение последовательности.

Базовая случайная величина вычислялась по формуле:

$$z_i = \frac{A_i}{2^r}.$$

Длина периода генератора — это количество элементов последовательности, после которого значения начинают повторяться. Для генератора «little-frog» период определяется выражением:

$$T = 2^{r-2}.$$

При $r = 16$:

$$T = 2^{14} = 16384.$$

В программе было сгенерировано 5000 псевдослучайных значений. Для проверки корректности работы датчика были вычислены выборочное математическое ожидание и выборочная дисперсия последовательности. Теоретические значения:

$$M(z) = 0.5, D(z) = \frac{1}{12} \approx 0.08333.$$

Выборочные значения определялись средствами библиотеки NumPy. На

рисунке 2 представлен вывод программы с результатами вычисления математического ожидания и дисперсии.

```
→ ~/Desktop/suai/MC/lab01 (.venv) (git:main) % py lab01.py
Теоретический период T = 16384
Выборочное мат. ожидание = 0.4945556121826172 (теор. 0.5)
Выборочная дисперсия = 0.08400835354047559 (теор. 0.08333333333333333)
```

Рисунок 2 — Вывод программы

Ожидается, что при увеличении количества генерируемых значений выборочные характеристики будут приближаться к теоретическим значениям. Для проверки свойства равномерности распределения псевдослучайных чисел интервал $[0; 1]$ был разбит на $K = 10$ равных частей. Далее была построена гистограмма распределения относительных частот попадания случайных величин в каждый интервал. Построение гистограммы выполнялось средствами библиотеки matplotlib. На рисунке 3 представлена гистограмма распределения псевдослучайных чисел.

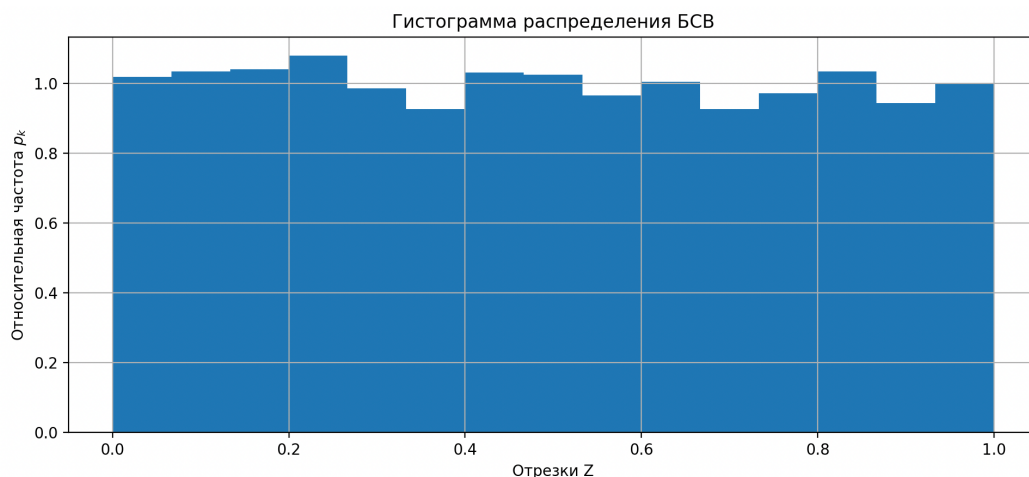


Рисунок 3 — Гистограмма распределения БСВ

По форме гистограммы можно сделать вывод, что распределение чисел близко к равномерному, так как высоты столбцов имеют приблизительно одинаковые значения.

Для проверки независимости случайных величин был вычислен

коэффициент корреляции:

$$\hat{R} = \frac{\sum (x_i - \bar{x})(y_i - \bar{y})}{\sqrt{\sum (x_i - \bar{x})^2 \sum (y_i - \bar{y})^2}}$$

Исследование проводилось для значений $s = 2, s = 5, s = 10$. Коэффициент корреляции вычислялся для различных значений длины выборки T [100; 3000) с шагом 100. На рисунке 4 представлены графики зависимости коэффициента корреляции $\hat{R}$ от длины последовательности T .

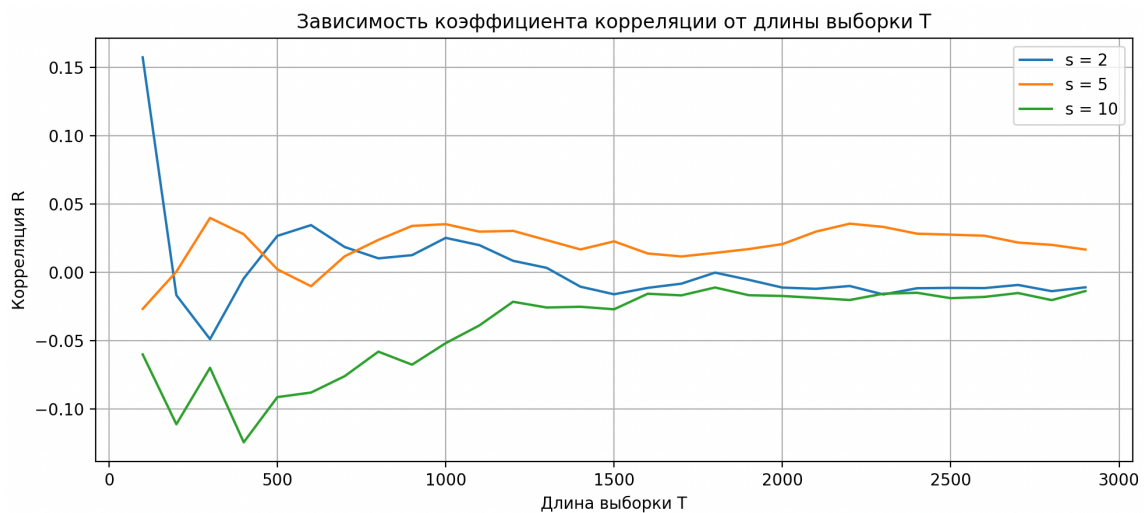


Рисунок 4 — Графики зависимости коэф. корреляции от T

По графикам видно, что значения коэффициента корреляции колеблются около нуля, что свидетельствует о слабой зависимости между элементами последовательности.

4 Выводы

В ходе лабораторной работы был реализован генератор базовой случайной величины «little-frog». Для выбранных параметров генератора теоретическая длина периода составила $T = 16384$, что удовлетворяет условию задания $T > 500$.

Выполнено исследование статистических свойств полученной последовательности псевдослучайных чисел. Полученные значения математического ожидания и дисперсии оказались близки к теоретическим значениям: $M(z) \approx 0,4946$ с отклонением $-0,00544$, $D(z) \approx 0,0840$ с отклонением $0,000675$. Расхождения объясняются конечным объемом

выборки ($N = 5000$) и особенностями псевдослучайного алгоритма.

Гистограмма распределения показала близость распределения к равномерному закону. Исследование коэффициента корреляции подтвердило слабую зависимость между элементами последовательности, что свидетельствует о приемлемом качестве генератора псевдослучайных чисел.

Следовательно, разработанный датчик обладает основными свойствами базовой случайной величины и может использоваться для моделирования псевдослучайных процессов.

ПРИЛОЖЕНИЕ

Листинг 1 — Листинг программы

```
import numpy as np
import matplotlib.pyplot as plt

def correlation(sequence, s, T):
    x = sequence[:T]
    y = sequence[s : s + T]
    return np.corrcoef(x, y)[0, 1]

r = 16 # число двоичных разрядов
MOD = 2**r # модуль
M = 65539 # множитель (взаимно простой с 2^r)
m = 3 # длина прыжка
A0 = 1 # начальное значение
M_jump = row(M, m, MOD) # прыжковый множитель

T_theory = 2 ** (r - 2) # теоретический период
N = 5000 # количество генерируемых чисел
K = 15 # количество интервалов для гистограммы

# генерация БСВ
A = np.zeros(N, dtype=np.uint64)
Z = np.zeros(N)
A[0] = A0

for i in range(1, N):
    A[i] = (A[i - 1] * M_jump) % MOD
    Z[i] = A[i] / MOD

# математическое ожидание и дисперсия
M_exp = np.mean(Z)
D_exp = np.var(Z)
M_theory = 0.5
D_theory = 1 / 12

print(f"Теоретический период T = {T_theory}")
print(f"Выборочное мат. ожидание = {M_exp} (теор. {M_theory})")
print(f"Выборочная дисперсия = {D_exp} (теор. {D_theory})")

fig, (ax1, ax2) = plt.subplots(2, 1, figsize=(10, 9))

# гистограмма распределения
ax1.hist(Z, bins=K, density=True)
ax1.set_title("Гистограмма распределения БСВ")
ax1.set_xlabel("Отрезки Z")
ax1.set_ylabel(r"Относительная частота $p_k$")
ax1.grid(True)

# коэффициент корреляции
T_values = np.arange(100, 3000, 100)
s_values = [2, 5, 10]
for s in s_values:
    R_values = []
    for T in T_values:
        R = correlation(Z, s, T)
        R_values.append(R)
```

```
ax2.plot(T_values, R_values, label=f"s = {s}")

ax2.set_title("Зависимость коэффициента корреляции от длины выборки T")
ax2.set_xlabel("Длина выборки T")
ax2.set_ylabel("Корреляция R")
ax2.grid(True)
ax2.legend()

plt.tight_layout()
plt.show()
fig.savefig("plots.png")
```